Django Debug Toolbar – Complete Notes (বাংলা)

1. What is Django Debug Toolbar?

Django Debug Toolbar হলো Django-এর একটি Third-party debugging tool, যা Django Web Application-এর Request, Response, Database Query, Execution Time এবং অন্যান্য debugging information দেখায়।

সহজ ভাষায়, Django Application-এর ভিতরে কী ঘটছে, তা ব্রাউজারে দেখার জন্য Django Debug Toolbar ব্যবহার করা হয়।

Example

তুমি যখন Django Website চালু করবে, তখন ব্রাউজারের ডান পাশে একটি Sidebar দেখতে পারবে।

এই Sidebar-এর মাধ্যমে তুমি জানতে পারবে:

- কোন Database Query Execute হয়েছে।
- কত সময়ে Page Load হয়েছে।
- কোন HTML Template ব্যবহার হয়েছে।
- কোন HTTP Request এসেছে।
- Django-এর কোন Settings ব্যবহার হচ্ছে।

মনে রাখবে: Django Debug Toolbar মূলত Development Environment-এ ব্যবহার করা হয়।

---

2. Why Do We Use Django Debug Toolbar?

Django Debug Toolbar ব্যবহার করার প্রধান কারণগুলো হলো:

1. **Debugging:** Application-এর সমস্যা খুঁজে বের করা।
2. **Performance Monitoring:** কোন কাজ করতে কত সময় লাগছে, তা দেখা।
3. **SQL Query Analysis:** Database Query পরীক্ষা করা।
4. **Template Inspection:** কোন Template Render হয়েছে, তা দেখা।
5. **Request Inspection:** HTTP Request ও Response সম্পর্কে তথ্য দেখা।
6. **Optimization:** Application-এর Performance উন্নত করা।

Example

ধরো, তোমার Django Application-এর একটি Page Load হতে ৩ সেকেন্ড সময় লাগছে।

Django Debug Toolbar ব্যবহার করে তুমি SQL Panel পরীক্ষা করতে পারবে। সেখানে ধীরগতির Database

Query থাকলে সেটি শনাক্ত করে Optimize করতে পারবে।

---

3. How to Install Django Debug Toolbar

প্রথমে তোমার Django Project-এর Virtual Environment Activate করো।

তারপর Terminal-এ নিচের Command চালাও:

```bash
pip install django-debug-toolbar
```

ইনস্টলেশন সম্পন্ন হলে `settings.py` ফাইলে কনফিগার করতে হবে।

---

4. Configure Django Debug Toolbar

Step 1: Add to `INSTALLED_APPS`

`settings.py` ফাইলে `INSTALLED_APPS`-এর মধ্যে `debug_toolbar` যোগ করো।

```python
INSTALLED_APPS = [
    # Default Django apps
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Third-party apps
    "debug_toolbar",
]
```

ব্যাখ্যা:

`debug_toolbar`-কে `INSTALLED_APPS`-এ যুক্ত করার মাধ্যমে Django-কে জানানো হয় যে Application-এ Debug Toolbar ব্যবহার করা হবে।

Step 2: Add Middleware

`settings.py` ফাইলের `MIDDLEWARE` List-এ নিচের Middleware যোগ করো।

```python
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",

    # Django Debug Toolbar
    "debug_toolbar.middleware.DebugToolbarMiddleware",
]
```

ব্যাখ্যা:

Middleware হলো Django-এর Request ও Response Processing System-এর একটি অংশ।

`DebugToolbarMiddleware` Request ও Response-এর বিভিন্ন তথ্য সংগ্রহ করে Debug Toolbar প্রদর্শনে সাহায্য করে।

সাধারণত এই Middleware-টি `MIDDLEWARE` List-এ যতটা সম্ভব শেষের দিকে রাখা হয়।

Step 3: Configure `INTERNAL_IPS`

`settings.py` ফাইলে নিচের Configuration যোগ করো:

```python
INTERNAL_IPS = [
    "127.0.0.1",
]
```

ব্যাখ্যা:

`127.0.0.1` হলো Localhost IPv4 Address।

Local Development Environment-এ নিজের কম্পিউটার থেকে Application Access করার সময় এই Configuration ব্যবহার করা হয়।

তবে মনে রাখবে, Debug Toolbar প্রদর্শনের জন্য `INTERNAL_IPS` ছাড়াও অন্যান্য শর্ত পূরণ হতে হয়।

Step 4: Configure `urls.py`

তোমার Project-এর মূল `urls.py` ফাইল খোলো।

উদাহরণ:

```python
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),

    # Django Debug Toolbar
    path("__debug__/", include("debug_toolbar.urls")),
]
```

ব্যাখ্যা:

```python
path("__debug__/", include("debug_toolbar.urls"))
```

এই URL Configuration Debug Toolbar-এর নিজস্ব URL-গুলো যুক্ত করে।

নোট: এটি Project-এর মূল `urls.py` ফাইলে যোগ করতে হবে। তোমার Project-এ আগে থেকেই `urlpatterns` থাকলে সেগুলো মুছে ফেলবে না।

---

5. How Does Django Debug Toolbar Work?

Django Debug Toolbar মূলত Django Application-এর Request ও Response Processing-এর সময় বিভিন্ন তথ্য সংগ্রহ করে এবং সেগুলো ব্রাউজারে দেখায়।

Working Process

```text
```

```
       User
         |
         v
    HTTP Request
         |
         v
  Django Middleware
         |
         v
      Django View
         |
         v
   Database Query
         |
         v
   Template Rendering
         |
         v
    HTTP Response
         |
         v
 Django Debug Toolbar
         |
         v
 Browser Sidebar
```

এটি একটি ধারণামূলক Diagram। বাস্তবে Debug Toolbar Middleware এবং বিভিন্ন Panel-এর মাধ্যমে তথ্য সংগ্রহ করে।

Example

ধরো, তোমার Django Application-এ একটি View আছে:

```python
from django.contrib.auth.models import User
from django.shortcuts import render


def home(request):
    users = User.objects.all()
```

```
    return render(
        request,
        "home.html",
        {"users": users},
    )
```

এই View-টি Database থেকে User-এর তথ্য সংগ্রহ করে।

Django Debug Toolbar-এর SQL Panel থেকে তুমি Query-সংক্রান্ত তথ্য দেখতে পারবে।

---

6. Django Debug Toolbar-এর গুরুত্বপূর্ণ Panels

Django Debug Toolbar-এর বিভিন্ন Panel-এর কাজ নিচে ব্যাখ্যা করা হলো।

6.1 History Panel

কাজ: পূর্ববর্তী Request-গুলোর Debugging তথ্য দেখতে সাহায্য করে।

উদাহরণ:

```text
/
/about/
/contact/
```

এখানে পূর্ববর্তী Request-এর তথ্য পাওয়া যেতে পারে।

6.2 Versions Panel

কাজ: Django এবং অন্যান্য গুরুত্বপূর্ণ Package-এর Version-সংক্রান্ত তথ্য দেখায়।

উদাহরণ:

```text
Django 6.1.2
```

এটি একটি উদাহরণমাত্র। তোমার Project-এ Installed Django Version আলাদা হতে পারে।

6.3 Time Panel

কাজ: Request Processing-এর সময় সম্পর্কে তথ্য দেখায়।

উদাহরণ:

```text
Total: 372.64ms
```

এর অর্থ, Toolbar-এ প্রদর্শিত Request-এর জন্য মোট সময় হিসেবে প্রায় ৩৭৩ মিলিসেকেন্ড দেখানো হচ্ছে।

এই সময়কে পুরোপুরি Browser-এর Page Load Time-এর সমান ধরে নেওয়া যাবে না।

6.4 Settings Panel

কাজ: Django Project-এর Settings-সংক্রান্ত তথ্য দেখায়।

উদাহরণ:

```python
DEBUG = True
```

এখানে `DEBUG`, `INSTALLED_APPS`, `MIDDLEWARE` এবং অন্যান্য Settings-এর তথ্য পাওয়া যেতে পারে।

সতর্কতা: Settings Panel-এ Sensitive Configuration প্রকাশিত হতে পারে। তাই Production Environment-এ Debug Toolbar চালু রাখা উচিত নয়।

6.5 Headers Panel

কাজ: HTTP Request ও Response-এর Header-সংক্রান্ত তথ্য দেখায়।

উদাহরণ:

```text
Content-Type: text/html
```

HTTP Headers-এর মাধ্যমে Client ও Server বিভিন্ন তথ্য আদান-প্রদান করে।

6.6 Request Panel

কাজ: বর্তমান HTTP Request সম্পর্কে তথ্য দেখায়।

এতে থাকতে পারে:

- Request Method
- GET Parameters
- POST Parameters
- Cookies
- Request Metadata

উদাহরণ:

```text
GET /
```

এখানে `GET` হলো HTTP Method এবং `/` হলো Request Path।

6.7 SQL Panel

কাজ: Django ORM-এর মাধ্যমে তৈরি হওয়া Database Query পরীক্ষা করতে সাহায্য করে।

উদাহরণ:

```sql
SELECT * FROM auth_user;
```

এই Query `auth_user` Table থেকে তথ্য সংগ্রহ করে।

SQL Panel থেকে সাধারণত নিচের তথ্যগুলো দেখা যায়:

- Executed SQL Queries
- Query Execution Time
- Query Count
- Query-এর বিস্তারিত তথ্য

কেন SQL Panel গুরুত্বপূর্ণ?

ধরো, একটি Page Load করার সময় একই ধরনের Query বারবার Execute হচ্ছে।

SQL Panel ব্যবহার করে সমস্যাটি শনাক্ত করতে পারবে এবং প্রয়োজনে Django ORM-এর `select_related()` বা `prefetch_related()` ব্যবহার করে Query-এর সংখ্যা কমাতে পারবে।

6.8 Static Files Panel

কাজ: বর্তমান Request-এর সঙ্গে সম্পর্কিত Static File ব্যবহারের তথ্য দেখায়।

Django-তে Static Files বলতে সাধারণত বোঝায়:

- CSS
- JavaScript
- Images
- অন্যান্য Static Assets

Screenshot-এ `0 files used` দেখা যাচ্ছে। এর অর্থ, ওই Request-এর জন্য Panel কোনো Static File ব্যবহারের তথ্য দেখায়নি।

6.9 Templates Panel

কাজ: কোন Template Render হয়েছে, তা দেখতে সাহায্য করে।

উদাহরণ:

```text
polls/home.html
```

এখানে `polls/home.html` হলো একটি Template-এর Path।

এখানে Template Context এবং Template Rendering-সংক্রান্ত তথ্যও পাওয়া যেতে পারে।

6.10 Alerts Panel

কাজ: কিছু সম্ভাব্য সমস্যা বা সতর্কতা সম্পর্কে তথ্য দেখাতে পারে।

এটি বিভিন্ন ধরনের Debugging সমস্যা শনাক্ত করতে সাহায্য করে।

তবে এটি সব ধরনের Error বা Bug শনাক্ত করতে পারবে, এমন নয়।

6.11 Cache Panel

কাজ: Django Cache ব্যবহারের তথ্য দেখায়।

উদাহরণ:

```python
from django.core.cache import cache
```

```
cache.set("username", "Faruk", timeout=60)

username = cache.get("username")

print(username)
```

এখানে:

- `cache.set()` Cache-এ তথ্য সংরক্ষণ করে।
- `cache.get()` Cache থেকে তথ্য সংগ্রহ করে।
- `timeout=60` অর্থ Cache Entry-এর মেয়াদ ৬০ সেকেন্ড।

Cache Panel থেকে Cache Operation-সংক্রান্ত তথ্য দেখা যেতে পারে।

6.12 Signals Panel

কাজ: Django Signals-এর মাধ্যমে পরিচালিত বিভিন্ন কার্যক্রম সম্পর্কে তথ্য দেখতে সাহায্য করে।

Django Signal ব্যবহার করে একটি নির্দিষ্ট ঘটনা ঘটলে অন্য অংশের Code Execute করা যায়।

উদাহরণ:

```python
from django.contrib.auth.models import User
from django.db.models.signals import post_save
from django.dispatch import receiver


@receiver(post_save, sender=User)
def user_created(sender, instance, created, **kwargs):
    if created:
        print("New user created")
```

এখানে User Model-এর কোনো Object প্রথমবার তৈরি হয়ে Save হওয়ার পরে `created=True` হলে Signal Handler Execute হয়।

Debug Toolbar-এর Signals Panel ব্যবহার করে সংশ্লিষ্ট Signal-সংক্রান্ত তথ্য পরীক্ষা করা যেতে পারে।

---

7. How to Hide Django Debug Toolbar

Method 1: Temporarily Hide

Toolbar-এর উপরে `Hide »`-এ ক্লিক করো।

এতে Sidebar লুকিয়ে যাবে।

তবে Debug Toolbar-এর Configuration পরিবর্তন হবে না।

Method 2: Disable Django Debug Toolbar

যদি Toolbar পুরোপুরি বন্ধ করতে চাও, তাহলে নিচের ধাপগুলো অনুসরণ করো।

Step 1: `settings.py` থেকে `INSTALLED_APPS`-এর এই Entry সরিয়ে দাও।

```python
"debug_toolbar",
```

Step 2: `MIDDLEWARE` থেকে এই Entry সরিয়ে দাও।

```python
"debug_toolbar.middleware.DebugToolbarMiddleware",
```

Step 3: Project-এর মূল `urls.py` থেকে এই URL সরিয়ে দাও।

```python
path("__debug__/", include("debug_toolbar.urls")),
```

এরপর Django Development Server Restart করো।

মনে রাখবে: Toolbar বন্ধ করার জন্য Package Uninstall করা বাধ্যতামূলক নয়।

---

8. How to Disable Django Debug Toolbar in Production

Django Debug Toolbar মূলত Development Environment-এর জন্য।

Production Environment-এ এটি চালু রাখলে Sensitive Debugging Information প্রকাশিত হতে পারে।

তাই Development এবং Production-এর Settings আলাদা রাখা ভালো।

উদাহরণ:

```python
DEBUG = False
```

Production Environment-এ শুধু `DEBUG = False` করাই যথেষ্ট নয়। Debug Toolbar-এর App, Middleware এবং URL Configuration-ও উপযুক্তভাবে বন্ধ রাখতে হবে।

একটি সাধারণ উদাহরণ:

```python
DEBUG = False

if DEBUG:
    INSTALLED_APPS += [
        "debug_toolbar",
    ]

    MIDDLEWARE += [
        "debug_toolbar.middleware.DebugToolbarMiddleware",
    ]
```

তবে এই উদাহরণে `INSTALLED_APPS` এবং `MIDDLEWARE`-এর Entry Conditional করা হয়েছে। `urls.py`-তেও Debug Toolbar-এর URL Pattern-কে একইভাবে উপযুক্ত শর্তের অধীনে রাখতে হবে।

উদাহরণ:

```python
from django.conf import settings
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += [
        path("__debug__/", include("debug_toolbar.urls")),
    ]
```

```

এভাবে Debug Toolbar-এর URL শুধু `DEBUG=True` হলে যুক্ত হবে।

**নোট:** Production-এ `DEBUG=False` রাখা এবং Sensitive Configuration সুরক্ষিত রাখা অত্যন্ত গুরুত্বপূর্ণ।

---

## 9. Django Debug Toolbar vs Django Admin

| বিষয় | Django Debug Toolbar | Django Admin |
|---|---|---|
| প্রধান কাজ | Debugging | Data Management |
| Database Query পরীক্ষা | হ্যাঁ | প্রধান উদ্দেশ্য নয় |
| Request Information | হ্যাঁ | প্রধান উদ্দেশ্য নয় |
| Model Data Management | না | হ্যাঁ |
| Development-এ ব্যবহার | হ্যাঁ | হ্যাঁ |
| Production-এ ব্যবহার | সাধারণত বন্ধ রাখা উচিত | নিরাপদভাবে কনফিগার করে ব্যবহার করা যায় |

### সহজভাবে মনে রাখো

- **Django Debug Toolbar:** Application-এর ভিতরে কী হচ্ছে, তা পরীক্ষা করার জন্য।
- **Django Admin:** Database-এর Model Data পরিচালনা করার জন্য।

---

## 10. Advantages of Django Debug Toolbar

Django Debug Toolbar ব্যবহারের সুবিধাগুলো হলো:

1. সহজে Debugging করা যায়।
2. Database Query পরীক্ষা করা যায়।
3. Query Execution Time দেখা যায়।
4. Template Rendering সম্পর্কে তথ্য পাওয়া যায়।
5. Request ও Response বিশ্লেষণ করা যায়।
6. Performance Bottleneck শনাক্ত করতে সাহায্য করে।
7. Development-এর সময় সমস্যা খুঁজে বের করা সহজ হয়।

---

## 11. Disadvantages of Django Debug Toolbar
```

কিছু সীমাবদ্ধতাও রয়েছে:

1. এটি অতিরিক্ত Debugging Information সংগ্রহ করে।
2. Debugging-এর জন্য অতিরিক্ত Processing Overhead তৈরি হতে পারে।
3. সব ধরনের Bug শনাক্ত করতে পারে না।
4. Production-এ চালু থাকলে Sensitive Information প্রকাশিত হতে পারে।
5. Toolbar প্রদর্শনের জন্য উপযুক্ত Request, Response এবং Configuration প্রয়োজন।

---

12. Important Interview Questions

Q1. What is Django Debug Toolbar?

Answer: Django Debug Toolbar is a third-party debugging tool that displays information about Django requests, SQL queries, templates, settings, and execution time.

Q2. Why do we use Django Debug Toolbar?

Answer: We use it to debug Django applications, inspect SQL queries, analyze request processing, and identify potential performance problems.

Q3. What is the purpose of the SQL Panel?

Answer: The SQL Panel displays database queries executed during a request and provides information such as query execution time.

Q4. What is the purpose of the Templates Panel?

Answer: The Templates Panel helps identify which templates were rendered during a request and inspect template-related information.

Q5. Should we use Django Debug Toolbar in production?

Answer: Generally, no. It is intended primarily for development because it may expose sensitive debugging information.

Q6. How can we hide Django Debug Toolbar?

Answer: We can click the `Hide` option to temporarily hide the toolbar or remove its configuration to disable it.

Q7. Is Django Debug Toolbar a built-in Django feature?

Answer: No. Django Debug Toolbar is a third-party package that must be installed and configured separately.

Q8. What is the purpose of `INTERNAL_IPS`?

Answer: `INTERNAL_IPS` specifies IP addresses considered internal by Django Debug Toolbar. It is one of the settings that can affect whether the toolbar is displayed.

---

13. Quick Revision

Term	Meaning
Django Debug Toolbar	Django Application Debugging Tool
Middleware	Request ও Response Processing-এর অংশ
SQL Panel	Database Query পরীক্ষা করে
Templates Panel	Render হওয়া Template-এর তথ্য দেখায়
Time Panel	Request Processing-এর সময় দেখায়
Headers Panel	HTTP Headers-এর তথ্য দেখায়
Request Panel	HTTP Request-এর তথ্য দেখায়
Cache Panel	Cache Operation-এর তথ্য দেখায়
Signals Panel	Django Signals-সংক্রান্ত তথ্য দেখায়
`INTERNAL_IPS`	Internal IP Address Configuration
`DEBUG = False`	Django-এর Debug Mode বন্ধ করে

Final Note

Django Debug Toolbar-এর মূল উদ্দেশ্য হলো Django Application-এর অভ্যন্তরীণ কার্যক্রম পর্যবেক্ষণ করে Debugging এবং Performance Analysis সহজ করা।

বিশেষ করে Django Backend Developer হিসেবে SQL Panel, Request Panel এবং Templates Panel ভালোভাবে শেখা গুরুত্বপূর্ণ।